

SCRIBBLE TO DIGITAL

Team ID: SWTID-2026-3152

Team Size: 5

Team Leader: AFRIN BANU S

Team member: SARANYA N

Team member: KOWSHIKA S

Team member: ALVIN ANTONY X

Team member: BALAJI S

Project Overview

Scribble to Digital is an AI-powered application designed to solve the common challenge of converting messy handwritten notes into clean, structured digital text. It not only digitizes text but also improves grammar and automatically extracts actionable tasks, saving users from manual rewriting.

Key Features

- **Image Enhancement:** The system uses OpenCV to convert images to grayscale, adjust brightness and contrast, and reduce noise to ensure better readability for low-quality handwriting.
- **Intelligent Text Correction:** It uses Generative AI to fix spelling errors, improve sentence structure, and reconstruct incomplete words using contextual understanding.
- **Automatic To-Do Extraction:** The AI detects task-oriented statements and organizes them into a separate, structured bullet-point list.
- **Export Options:** Users can download their processed notes and task lists as TXT, CSV, PDF, or Word documents.

Core Technologies

Component	Technology	Description
Frontend UI	Streamlit	Provides an interactive web interface for users to upload images and view processed results.
Image Preprocessing	OpenCV	Enhances uploaded images (grayscale, noise reduction, thresholding) to improve text recognition accuracy.
Text Extraction	Tesseract / EasyOCR	The OCR engine that converts the enhanced handwritten images into raw, machine-readable text.
Generative AI	Google Gemini 2.5 Flash	Processes the raw text to correct grammar, reconstruct words, format paragraphs, and extract action items.
Backend Logic	Python	The core programming language powering the application and integrating all modules.

How It Works (The User Workflow)

1. **Upload:** A user uploads a photo of their handwritten notes (JPG, PNG, or JPEG) through the Streamlit web interface.
2. **Process Image:** The backend automatically enhances the image clarity so the OCR engine can accurately read the raw text.
3. **Apply AI:** The raw, often messy text is sent to the Gemini AI model, which cleans up the grammar, formats the notes, and pulls out specific tasks.
4. **View & Download:** The user views the clean digital text and extracted to-do list on their screen and can download the results for their records.

System Requirements & Setup

To run this project, the following prerequisites are required:

- Python 3.8 or higher installed.
- A Google AI Studio account to generate a Gemini API Key.
- Tesseract OCR installed on the host machine.
- Required Python libraries: streamlit, opencv-python, pytesseract, google-generativeai, pillow, and python-dotenv.

Reliability & Testing

The system has been comprehensively tested to handle various real-world scenarios:

- **Varying Image Quality:** It successfully processes clear handwriting, medium-quality images with slight shadows, and poor-quality blurry text.
- **Edge Cases Handled:** The application includes error handling to prevent crashes from blank image uploads, non-text images, or API connection failures.

Detailed Technical Architecture

The application uses a modular architecture to separate the user interface, image processing, and AI logic.

- **Frontend (Streamlit):** The UI is built with Streamlit, featuring a sidebar for navigation between a "Home" landing page and a "Process Notes" generator page. The interface includes file uploaders, loading spinners for user feedback, and separate scrollable text areas for the cleaned notes and to-do lists.

- **Backend (Flask Integration):** The backend logic handles input validation (checking file types and size limits), error handling for API or OCR failures, and structured JSON response formatting.

Modular File Structure:

- `app.py`: The main application runner.
- `utils/ocr_processor.py` (or `services/ocr_service.py`): Handles image preprocessing and text extraction.
- `utils/ai_processor.py` (or `services/ai_service.py`): Manages the integration with the Gemini API for contextual correction.
- `utils/file_export.py` (or `services/export_service.py`): Manages the generation of TXT, PDF, Word (.docx), and CSV files.

The OpenCV Image Processing Pipeline

Before the AI can read the text, the raw image must be cleaned up so the OCR engine (Tesseract or EasyOCR) can parse it accurately. The specific steps in the code include:

1. **Grayscale Conversion:** Converting the uploaded color image to grayscale (`cv2.cvtColor`).
2. **Brightness & Contrast Adjustment:** Enhancing the image to make faded ink more visible.
3. **Noise Removal:** Cleaning up background artifacts or paper texture.
4. **Thresholding:** Applying binary thresholding (`cv2.threshold`) to turn the image into stark black and white, which is optimal for OCR engines.

Comprehensive Testing & Edge Cases

The developers stress-tested the application to ensure it handles various real-world scenarios gracefully:

- **Handwriting Variations:** It was tested against clear handwriting (accurate extraction), medium-quality handwriting with slight shadows (AI successfully guessed context), and poor-quality blurry text (where the image enhancement was crucial for partial recovery).
- **Content Variations:** The system was tested on informational notes with no tasks to ensure it accurately returns a "No tasks detected" state, as well as notes containing heavily formatted bullet points.
- **Performance Optimizations:** API token usage was reduced by improving prompt clarity, and the system was configured to prevent automatic

reprocessing if the user refreshes the Streamlit page.

- **Error Handling:** Edge cases like blank images, non-text images, OCR failures, and internet connectivity dropouts are caught and display user-friendly error messages rather than crashing the server.

Detailed AI Processing & Prompt Engineering

The system's intelligence relies on how the **Google Gemini 2.5 Flash** model is instructed. The prompt is not just a request but a rigid set of system commands designed to force structured output, essential for the UI display (as seen in image_0.png).

Exact System Prompt

The model is configured with this system instruction, sent before any user data:

Plaintext

SYSTEM: You are a "Handwritten Note Optimizer".

1. Read the provided OCR text, which may be messy and contain errors.
2. Correct spelling and grammar.
3. Formatting: Output the result in a clean, professional format.
4. Data Extraction (IMPORTANT): Identify any specific tasks or action items and list them separately as a bulleted list.
5. Response Format: Your entire response must be a single, valid JSON object with two keys:
 - "cleaned_notes": A single string containing the well-formatted, corrected text.
 - "extracted_tasks": A list of strings, where each string is a single actionabl

Data Flow Example

- **User Upload (Image):** A photo with the handwritten text: *"Call bob about phoenix budget. Need numbers soon. Remembr to send email to team A."*
- **OpenCV + OCR Output (Raw Text):** *"Call bb abut phnx budgt. Ned nmbers sn. Remembr to send emil to tam A."*
- **JSON Response (from Gemini):**

JSON

```
{  
  "cleaned_notes": "Call Bob about the Phoenix budget. The numbers are  
needed soon. Remember to send an email to Team A.",  
  "extracted_tasks": [  
    "Call Bob about Phoenix budget",  
    "Gather budget numbers",  
    "Send email to Team A"  
  ]  
}
```

Implementation of File Export Service

The documentation mentions exports to PDF, Word, TXT, and CSV. Here is how the application builds these files based on the JSON output above:

- **TXT (Plan Text):** A simple concatenation of "CLEANED NOTES:\n" + cleaned_notes + "\n\nTASKS:\n" + "\n".join(extracted_tasks).
- **DOCX (using python-docx):** A document object (Document()) is created. It adds a Heading 1 for "Notes", then adds the cleaned_notes string. It then adds another Heading 1 for "Tasks" and uses a loop to add each item from extracted_tasks as a paragraph with the 'List Bullet' style (similar to the diagram in image_4.png).
- **PDF (using reportlab):** A drawing surface (Canvas) is initialized. It draws the main title, then uses Paragraph objects (handling automatic text wrapping) to place the cleaned notes and task list at specific coordinates on the page.
- **CSV (Structured):** Uses the Python csv module to create a standard comma-separated file. It writes two headers (Type, Content), followed by one row for 'Cleaned Notes' and multiple rows, one for each 'Task' item, enabling easy import into spreadsheet software or task managers.

Updated Technical Sequence Diagram

The following diagram provides a granular look at the data structure flow and specific method interactions:

Code snippet

sequenceDiagram

participant User

participant StreamlitUI as Streamlit UI

participant ImageService as ocr_processor.py (OpenCV+OCR)

participant GeminiAPI as Gemini 2.5 API

participant ExportService as file_export.py (PDF/DOCX/CSV)

Note over User, StreamlitUI: Step 1: Upload

User->>StreamlitUI: Uploads Image file (PNG/JPG)

Note over StreamlitUI, ImageService: Step 2: Image Processing

StreamlitUI->>ImageService: get_clean_text(image_file)

ImageService->>ImageService: process_image(grayscale, denoising, thresholding)

ImageService->>ImageService: run_tesseract(enhanced_image)

ImageService-->>StreamlitUI: returns raw_text_string

Note over StreamlitUI, GeminiAPI: Step 3: AI Structuring

StreamlitUI->>GeminiAPI: generate_content(raw_text_string)

GeminiAPI->>GeminiAPI: Applies strict System Prompt

GeminiAPI-->>StreamlitUI: returns structured JSON_object (cleaned_notes, extracted_tasks)

Note over StreamlitUI: Step 4: Display Result

StreamlitUI->>StreamlitUI: Render UI (image_0.png) with JSON data

Note over User, ExportService: Step 5: Export

User->>StreamlitUI: Clicks "WORD" or "PDF"

StreamlitUI->>ExportService: create_export_file(file_type, JSON_object)

ExportService->>ExportService: Build document (e.g., using reportlab/python-docx logic)

ExportService-->>User: Downloads structured document

This screen visualizes the entire workflow in one view:

1. **Sidebar:** Navigation and file upload status.
2. **Input (Left):** A preview of the original, messy handwritten note on lined paper.
3. **Processing Status:** Confirms that OpenCV has enhanced the image.
4. **Output (Right):** Two separate boxes showing the clean, digitally typed text and the automatically extracted "To-Do List" with checkmarks.
5. **Export (Bottom):** Clearly labeled download buttons for all supported formats (PDF, Word, TXT, CSV).

Dashboard

localhost:8501

NAVIGATION

Home

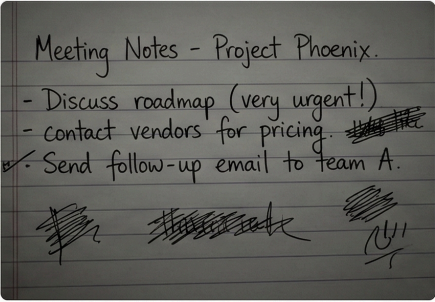
Process Notes (Selected)

Image Upload

uploaded: image_0.jpg

GENERATOR PAGE: Convert Your Notes

Original Handwritten Note (Preview)



Status: Image Enhanced Successfully 

Processed Digital Results

Cleaned & Formatted Text (Notes)

Meeting Notes - Project Phoenix:
Discuss the project roadmap (very urgent).
Contact vendors for pricing information.
Send a follow-up email to Team A.

Extracted Tasks (To-Do List)

 Discuss roadmap (very urgent)

 Contact vendors for pricing

 Send follow-up email to Team A

EXPORT YOUR NOTES

 PDF

 WORD

 TXT

 CSV